
Prédiction des retards des transports d'Île-de-France

Projet de data science appliquée aux mobilités urbaines

Christophe Mercier

Enzo Buisine

Mars 2026

Table des matières

1	Introduction	3
1.1	Contexte	3
1.2	Choix des données	3
2	Construction du dataset	3
2.1	Source principale : API PRIM	3
2.2	Ajout de la feature Occupation	4
2.3	Ajout de la feature Météo	4
2.4	Ajout de la feature Incidents	4
2.5	Fusion des sources	4
3	Feature engineering	5
3.1	Choix de la variable cible	5
3.2	Ajout de variables temporelles	6
3.3	Gestion des différences de valeurs entre opérateurs	7
3.4	Encodage des variables catégorielles	7
3.5	Renommage des variables	8
4	Analyse du dataset final	8
5	Data cleaning	9
5.1	Gestion des valeurs manquantes pour les transports	10
5.2	Répartition des données	12
6	Analyse exploratoire	13
6.1	Matrice de corrélation	13
6.2	Régression linéaire explicative	14
7	Tuning des hyperparamètres	14
7.1	Choix de la stratégie de validation	14
7.2	Limites du fine-tuning	15
8	Classification binaire	16
8.1	Choix des modèles	16
8.1.1	Régression logistique (baseline)	16
8.1.2	Arbre de décision	16
8.1.3	Random Forest (Forêt aléatoire)	17
8.1.4	Gradient Boosting / XGBoost	17
8.2	Choix des métriques	18

8.3	Résultats	18
8.3.1	Courbes ROC	18
8.3.2	Matrices de confusion	19
8.3.3	Tableau récapitulatif	21
8.3.4	Analyse des résultats	21
9	Régression	22
9.1	Choix des modèles	22
9.1.1	Régression linéaire (baseline)	22
9.1.2	Arbre de décision (régresseur)	23
9.1.3	Random Forest Regressor	23
9.1.4	Modèles de boosting	23
9.2	Choix des métriques	24
9.3	Résultats	24
9.3.1	Analyse des résultats	25
10	Conclusion	27
11	Pistes d'amélioration	28
11.1	Extension à l'ensemble des modes de transport	28
11.2	Exploration de modèles plus complexes	28
11.3	Enrichissement et diversification du dataset	28

1 Introduction

1.1 Contexte

Dans ce projet, nous nous intéressons à un phénomène qui concerne tous les habitants d’Île-de-France : les retards dans les transports. Que ce soit le métro, le bus, le RER ou les Transilien, les retards affectent quotidiennement des millions d’usagers et peuvent rendre leur quotidien plus difficile. Qui n’a jamais eu l’impression de devenir fou à cause des retards dans les transports ?

C’est dans ce contexte que nous nous sommes demandé si l’on pouvait, d’une part, **prédire si un train va arriver en retard** et, d’autre part, **estimer l’amplitude de ce retard** en secondes. La problématique est donc double : peut-on prédire si un retard va survenir, et si oui, de combien ? Cela permettrait à un voyageur de s’adapter en conséquence et de ne pas courir inutilement pour attraper son train.

1.2 Choix des données

Nous avons d’abord analysé différents jeux de données disponibles sur le portail PRIM. Nous nous sommes rapidement orientés vers l’API PRIM :

`https://prim.iledefrance-mobilites.fr/en/apis/idfm-ivtr-requete\_globale`

Cette API fournit des données en temps réel au format **GTFS-RT**, contenant notamment :

- l’heure théorique d’arrivée à un arrêt,
- l’heure effective (ou estimée) d’arrivée,
- la ligne, l’opérateur, le terminus et l’arrêt concerné.

Nous avons également envisagé l’API Navitia, qui fournit aussi des données en temps réel. Malheureusement, leur offre commerciale à 5 000 € par mois ne correspondait pas à notre budget, et leur offre Freemium a été arrêtée avant que nous puissions en bénéficier.

2 Construction du dataset

2.1 Source principale : API PRIM

L’API PRIM constitue la colonne vertébrale de notre jeu de données. Elle nous fournit, pour chaque passage de véhicule, les informations temporelles nécessaires au calcul du retard (différence entre l’heure prévue et l’heure réelle d’arrivée, exprimée en secondes), ainsi que diverses informations sur la position du train, sa ligne et sa destination.

2.2 Ajout de la feature Occupation

Pour enrichir le dataset, nous avons ajouté l'occupation estimée d'un train à son arrivée dans une station. Pour ce faire, nous avons utilisé un fichier `occupation.csv` qui, pour un *stopID* précis et une tranche horaire donnée, fournit le nombre d'entrées dans la station — proxy du niveau d'occupation. Cela nous permet d'estimer le nombre de personnes montant dans le train, ce qui peut provoquer des retards supplémentaires en raison du temps de montée, voire des incidents liés à la foule aux heures de pointe.

2.3 Ajout de la feature Météo

Après avoir constaté les importantes perturbations causées par des conditions climatiques défavorables — notamment en février, où quasiment tous les transports ont été interrompus — nous avons décidé d'intégrer des variables climatiques (neige, vent, pluie) à notre dataset. Pour ce faire, nous utilisons l'API :

`https://api.open-meteo.com/v1/forecast`

Cela nous permet d'obtenir les prévisions météo associées à chaque observation et d'étudier leur corrélation avec les retards, ainsi que l'impact réel des conditions climatiques sur le trafic.

2.4 Ajout de la feature Incidents

Nous avons enfin ajouté une feature relative aux incidents sur les lignes (travaux, accidents, problèmes matériels). Pour ce faire, nous avons interrogé l'API :

`https://prim.iledefrance-mobilites.fr/en/apis/idfm-disruptions\_bulk`

Cette API recense tous les messages diffusés par catégorie (travaux, incidents, etc.). Nous avons ensuite associé chaque message aux lignes concernées, en ajoutant une colonne `categorie_alerte` dans le dataset final. Il est à noter que tous les messages ne sont pas nécessairement synonymes de retard : des travaux ou un changement de terminus peuvent survenir sans engendrer de perturbations notables.

2.5 Fusion des sources

Pour relier les différents datasets entre eux, nous utilisons :

- le `stopID` et le `lineID` pour l'occupation,
- la date et l'heure pour la météo,
- le `lineID` pour les incidents.

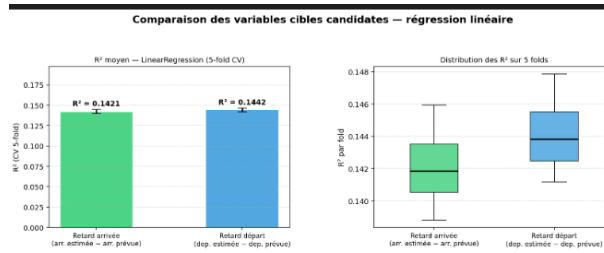


FIGURE 1 – Comparaison R^2

3 Feature engineering

Après la collecte de données, il nous reste à choisir quelles features doivent être retenues et lesquelles doivent être écartées, mais aussi comment calculer notre variable cible.

3.1 Choix de la variable cible

Nous avons deux possibilités pour construire notre variable de retard :

- Le retard entre l’heure d’arrivée prévue et l’heure d’arrivée estimée.
- Le retard entre l’heure de départ prévue et l’heure de départ estimée.

Ces deux features ne mesurent pas la même chose : l’heure d’arrivée correspond au moment où le train entre à quai, tandis que l’heure de départ correspond au moment où le train ferme ses portes et quitte la gare.

Pour décider laquelle utiliser, nous avons appliqué une régression linéaire sur chacune des deux variables et comparé les valeurs de R^2 obtenues. Nous avons ainsi obtenu : un R^2 de 0,1442 pour le départ contre 0,1421 pour l’arrivée. Même si l’écart est minime, nous avons décidé de retenir le départ comme métrique, car en plus de son score légèrement supérieur, c’est la variable la plus représentative du point de vue de l’usager : c’est l’horaire de départ qui indique à un voyageur quand son train va effectivement quitter la station. Il lui suffit d’arriver avant cet horaire pour être dans les temps, et c’est également cet horaire qui l’intéresse pour savoir quand il partira de la station.

Ce résultat nous a conduits à retenir le *retard au départ* comme variable cible pour la régression.

Pour construire la variable de classification binaire, nous avons utilisé un seuil simple : tout retard supérieur à 300 secondes (5 minutes) est considéré comme un train en retard. Ce seuil est celui retenu par la RATP pour définir officiellement un retard [?].

Proportion de trains en retard
(seuil RATP : 300 s — 1,536,446 passages)

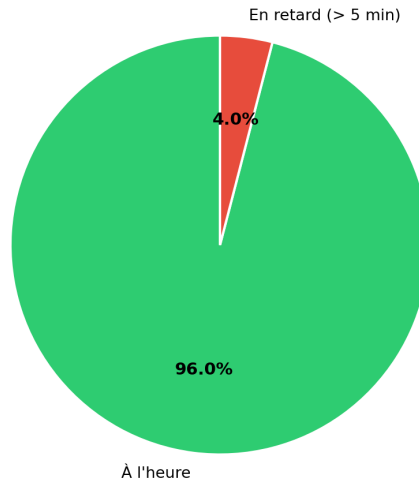


FIGURE 2 – Part des trains en retard

NB : ces proportions correspondent aux valeurs affichées par la RATP dans ses derniers rapports sur le nombre de trains en retard sur le réseau. Nos données sont donc concordantes avec les statistiques officielles¹. Nous avons donc des statistiques proches de la réalité.

En analysant cette répartition, nous avons constaté que, malgré la mauvaise réputation de la RATP, la majorité des trains arrivent à l'heure. Ce déséquilibre des classes risquait d'introduire un biais de classification : un modèle naïf pourrait obtenir un score artificiellement élevé en prédisant systématiquement « à l'heure ». Nous avons donc décidé d'utiliser le paramètre `class_weight='balanced'` afin de rééquilibrer les poids des classes et d'éviter ce biais.

3.2 Ajout de variables temporelles

Pour représenter au mieux les fréquences, nous ajoutons la tranche horaire du train, récupérée directement dans l'horaire du passage. Celle-ci nous aide fortement, car certains horaires sont plus propices aux retards : plus de risques d'incidents voyageurs, mais aussi plus de risques de propagation du retard en raison de la forte fréquence pendant les heures de pointe, définies par la RATP comme étant entre 7 h–9 h et 17 h–20 h. Nous les distinguons des autres dans 5 catégories (nuit, creux matin, pointe matin, creux après-midi, pointe soir).

Nous prenons également en compte les jours fériés, car même s'ils tombent en semaine,

1. <https://presse.iledefrance-mobilites.fr/ponctualite-de-lannee-2024-cinq-axes-et-le-t12-dedommag?lang=fr>

le trafic y est généralement différent : moins de voyageurs pressés, et un trafic du matin et du soir plus étalé dans le temps. Cette variable n'aura toutefois pas d'effet sur nos prédictions, car aucune donnée n'a été collectée pendant un jour férié.

Les vacances scolaires sont également prises en compte, étant donné que les habitudes de déplacement diffèrent sensiblement pendant ces périodes : la fréquentation est plus faible aux heures de pointe habituelles et le trafic est globalement plus étalé sur la journée.

3.3 Gestion des différences de valeurs entre opérateurs

Un des premiers problèmes auxquels nous avons été confrontés est l'hétérogénéité des valeurs entre opérateurs. En effet, le réseau Île-de-France Mobilités étant opéré par plusieurs prestataires (SNCF, RATP, Keolis, etc.), ceux-ci renvoient chacun les données dans le même format GTFS-RT, mais sans règle commune sur les valeurs elles-mêmes.

Par exemple, pour le sens du trajet, on peut rencontrer les valeurs : `A`, `R`, `Aller`, `Retour`, `outbound`, `inbound`, selon l'opérateur. Nous avons donc décidé d'unifier ces valeurs en les mappant sur 0 et 1.

3.4 Encodage des variables catégorielles

Pour des variables comme les stations, les terminus, etc., nous avons opté pour du *target encoding*, dans le but d'améliorer nos prédictions. Le target encoding consiste à remplacer chaque modalité d'une variable catégorielle par la moyenne de la variable cible pour cette modalité (par exemple, le RER B aurait une valeur correspondant à 300 s de retard moyen à Châtelet).

Pour éviter toute fuite de données (*data leakage*), nous utilisons la classe `TargetEncoder` de `scikit-learn`, appliquée aux stations et aux lignes. En effet :

- Un retard sur une ligne tend à se propager à l'ensemble de cette ligne.
- Chaque station est associée à un retard moyen qui lui est propre.

Afin de garantir la reproductibilité des résultats, nous utilisons un pipeline K-Fold ($k = 5$) de la forme :

```
pipeline = Pipeline([
    ('encoder', TargetEncoder(smooth='auto')),
    ('model', ModelClass())
])

kf = KFold(n_splits=5, shuffle=True, random_state=42)
cross_val_score(pipeline, X, y, cv=kf)
```

L'intérêt d'utiliser un objet `KFold` explicite plutôt que de passer simplement `cv=5` est

de garantir une reproductibilité parfaite grâce au paramètre `random_state`, indépendamment de l'environnement d'exécution. De plus, les données sont triées préalablement avant l'encodage afin de s'assurer qu'il n'y ait aucun leakage.

Notons qu'il n'y a pas de problème de stations partagées entre plusieurs lignes, chaque opérateur utilisant des codes de stations distincts. Les données sont également triées par ordre chronologique : l'objectif est de ne pas inclure dans l'entraînement des trains ayant subi la propagation d'un retard, ce qui donnerait des indices sur le retard des trains suivants.

Nous avons réalisé du *one-hot encoding* pour les variables temporelles, c'est-à-dire les jours de la semaine et la période de la journée, afin d'éviter que les modèles ne prédisent simplement grâce aux données les plus récentes. Cela nous a donné 7 variables pour les jours et 8 pour les catégories d'alerte (« aucune », « grève », « incident », « travaux », « météo », « retard », « voyageur », « autre »), ainsi que 9 pour les groupes météorologiques (« ensoleillé », « nuageux », « brouillard », « pluie », « neige », « averses », « orage », « autre », « inconnu »). Cela nous permet d'obtenir un encodage fin avec des coefficients adaptés à chaque valeur.

3.5 Renommage des variables

Pour bien distinguer les variables encodées des variables brutes, nous les renommons :

- `station` → `station_encoded`
- `ligne` → `ligne_encoded`
- `terminus` → `terminus_encoded`

Ces trois variables sont target-encodées : chacune est remplacée par la moyenne du retard observé pour la modalité correspondante.

4 Analyse du dataset final

Le dataset final contient les features suivantes :

Feature	Description
<code>retard_sec</code>	Variable cible — retard en secondes
<code>direction_ref</code>	Référence de la direction
<code>terminus_encoded</code>	Terminus encodé
<code>station_encoded</code>	Station encodée
<code>heure_tranche</code>	Tranche horaire
<code>mois</code>	Mois de l’observation
<code>jour_ferie</code>	Indicateur jour férié (0/1)
<code>est_vacances</code>	Indicateur vacances scolaires (0/1)
<code>precipitation</code>	Précipitations (mm)
<code>snowfall</code>	Chutes de neige (cm)
<code>wind_speed</code>	Vitesse du vent (km/h)
<code>temperature</code>	Température (°C)
<code>occupation</code>	Niveau d’occupation estimé
<code>nom_ligne</code>	Nom de la ligne
<code>jour_semaine</code>	Jour de la semaine
<code>periode_journee</code>	Période de la journée
<code>meteo_groupe</code>	Groupe météorologique
<code>categorie_alerte</code>	Catégorie d’incident

L’intérêt de ce dataset est d’être directement relié à des sources en temps réel, facilement extensible, et d’associer plusieurs features quantitatives pertinentes.

5 Data cleaning

L’une des premières difficultés rencontrées a été la gestion des données manquantes. En effet, le réseau étant opéré par plusieurs transporteurs, ceux-ci ne fournissent pas toujours toutes les informations, ce qui génère de nombreuses valeurs NaN.

Un problème plus structurel est que beaucoup de transports ne fournissent pas d’heure d’arrivée prévue : les métros, bus et trams ne fonctionnent pas selon des horaires fixes, mais selon une fréquence relative au train précédent. Ce problème touche également les bus et certains trams, bien que pas tous (le T9, par exemple, dispose d’horaires planifiés). Les bus sont particulièrement problématiques car, en plus de fonctionner par fréquences théoriques, ils sont extrêmement dépendants du trafic routier, des déviations et des accidents, ce qui les rend très difficiles à modéliser. Nous avons donc décidé de les exclure de notre dataset. Pour ce faire, nous avons récupéré les identifiants de toutes les lignes de RER, métro, tram et Transilien (lignes U, J, P, etc.) et supprimé le reste. Nous avons également exclu le Funiculaire de Montmartre ainsi que celui de Créteil, qui fonctionnent en continu et ne sont soumis qu’aux retards causés par des incidents techniques — sans lien

avec les phénomènes de montée/descente ou de propagation de retard que nous cherchons à modéliser.

5.1 Gestion des valeurs manquantes pour les transports

Après une analyse de nos données, nous nous sommes rendu compte qu'une grande partie des informations manquaient de valeur, que ce soit dans les métros ou même dans les trams. En comparaison, voici différents graphiques sur les valeurs manquantes en fonction du type de transport.

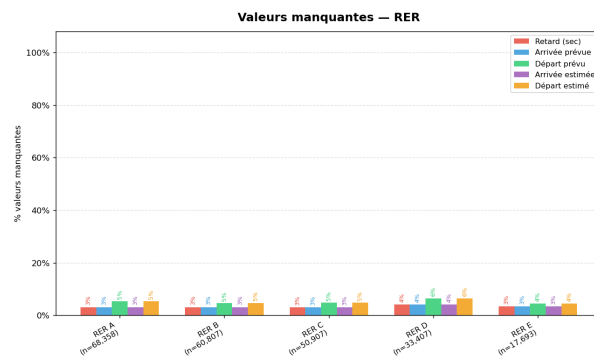


FIGURE 3 – Valeurs manquantes sur les différentes catégories pour les RER

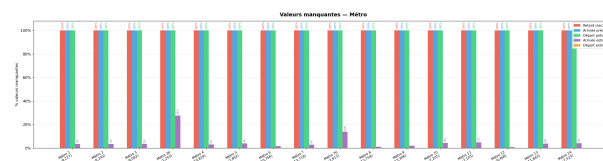


FIGURE 4 – Valeurs manquantes sur les différentes catégories pour le métro

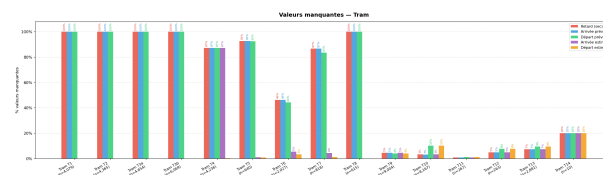


FIGURE 5 – Valeurs manquantes pour les trams

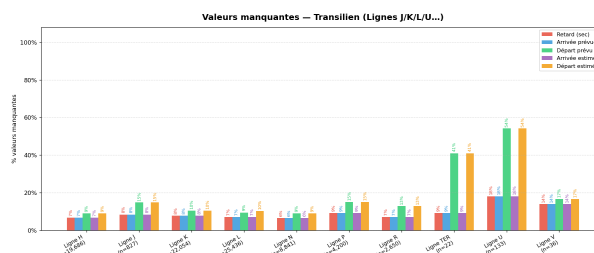


FIGURE 6 – Valeurs manquantes pour les Transilien

On constate que les Transilien et les RER sont facilement exploitables, avec un taux maximal de 10 % de valeurs manquantes pouvant être supprimées sans occulter la majorité du dataset. En revanche, ce n'est pas le cas pour les métros et les trams, qui présentent des taux proches de 100 % pour nombre d'entre eux : ces lignes ne disposent que des horaires estimés, mais pas des horaires prévus.

Ces difficultés nous ont fait comprendre qu'il fallait trouver une solution pour gérer ce problème. Nous avons d'abord envisagé de faire simplement une imputation en remplaçant les valeurs par la moyenne des retards théoriques de chaque ligne d'après les rapports d'IDFM. Cependant, étant donné que près de 70 % des valeurs sont manquantes sur certaines lignes (et même 100 % sur d'autres), cela aurait entraîné des modèles prédisant toujours la même chose, conduisant à de mauvais scores. Qui plus est, même si nous avions essayé de transposer les retards des RER sur les autres transports, cela aurait introduit des données erronées : les RER diffèrent des autres lignes tant par leur fréquence, le nombre de passagers transportés, que par les distances parcourues (avec près de 60 km de voies pour le RER A entre ses deux branches).

Les transports ne disposant pas d'horaires théoriques fixes, nous avons exploré plusieurs approches pour calculer leur retard :

1. **Récupération d'un fichier d'horaires RATP.** Ce fichier ne contient pas d'identifiant de véhicule utilisable : celui renvoyé par la RATP en cours de trajet est dynamique, dépendant du système, et la RATP elle-même déconseille de l'analyser. Par conséquent, un train prévu à 10 h 30 qui arrive à 10 h 40 ne serait pas détecté comme en retard, ce qui introduit un biais évident. De plus, la propagation d'un retard ferait considérer l'ensemble des trains suivants comme en retard, alors que ce n'est pas ainsi que la RATP le mesure.
2. **Exclusion de ces données.** Cette option préserve la qualité des données mais réduit significativement la taille du dataset, tout en nous permettant de nous limiter à une approche sur un type de transport en particulier en prenant bien en compte ses spécificités.
3. **Calcul de la différence par rapport au dernier passage.** En utilisant le dernier train passé à la même station (*stop point*), on calcule la différence entre son heure d'arrivée réelle et l'intervalle théorique attendu. Cette approche, bien que possible, comporte plusieurs problèmes, comme le volume de données à calculer et le temps de traitement.

Au vu de ces options, nous nous sommes orientés vers la deuxième approche, pour plusieurs raisons. La première méthode n'aurait été qu'une approche temporelle et n'aurait donné qu'une estimation approximative, pouvant causer des distorsions et des valeurs erronées. La troisième méthode s'est quant à elle révélée insuffisante lorsque nous l'avons testée :

l'API ne renvoie les données que lorsque les trains sont à l'arrêt sur le quai et non en déplacement (ce qui est compréhensible pour la connexion des trains, mais aussi parce que monitorer tous les transports aurait renvoyé un volume de données énorme). Cela introduit un problème : pour récupérer les données, il faut réussir à capter constamment les informations des trains, ce qui nécessite de maintenir la collecte avec une forte fréquence d'appel à l'API. Or, nous épuisions notre quota de consommation extrêmement vite sans obtenir de véritables résultats (un appel par minute, sachant que dans certaines stations les trains restent à peine une minute, sans compter les délais de l'API). Cela nous a fait abandonner cette approche pour les métros.

Pour les trams, les différences de contexte posaient un problème similaire : certains trams circulent dans des zones reculées où le trafic routier est plus faible qu'à Paris (par exemple les lignes T3), ce qui rend les retards non comparables. Nous avions de surcroît des problèmes où certaines lignes ne disposaient même pas des valeurs estimées pour leur arrivée en station, ce qui rendait la tâche encore plus ardue.

5.2 Répartition des données

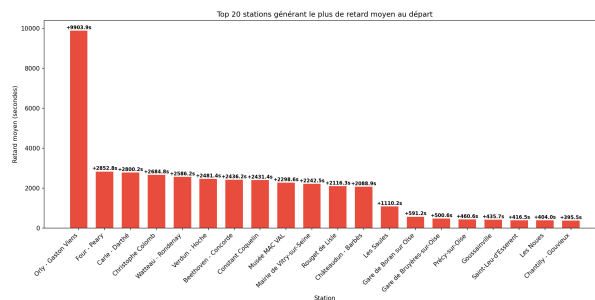


FIGURE 7 – Retards observés sur les principales stations de tram

Ce graphique illustre les retards relevés sur les stations de tram pendant la période de collecte. Les valeurs observées apparaissent très faibles, voire nulles, pour la grande majorité des passages. Cette distribution atypique s'explique directement par le contexte de collecte : au moment où les données ont été enregistrées, aucun incident majeur n'affectait les lignes de tram concernées. En l'absence de perturbations (travaux, incidents voyageurs, problèmes matériels), les trams circulent généralement à l'heure ou avec un écart très marginal, ce qui rend la tâche de prédiction peu informative sur ces périodes calmes. Ce constat illustre une limite fondamentale de toute collecte de données en temps réel : les performances des modèles sont intrinsèquement liées aux conditions prévalant au moment de l'enregistrement, et une période sans incident ne permet pas d'apprendre à prédire les situations dégradées.

Afin d'éviter un surapprentissage sur certaines tranches horaires, nous avons veillé à

conserver une répartition équitable du nombre de trains par tranche horaire (en tenant compte du fait qu'il y a naturellement moins de trains la nuit). L'objectif est de garantir que le modèle soit exposé à une distribution représentative des différentes périodes de la journée.

6 Analyse exploratoire

Une analyse rigoureuse des données est fondamentale avant toute modélisation. Peu importe la qualité du modèle : de mauvaises données en entrée produiront de mauvais résultats en sortie (*garbage in, garbage out*).

6.1 Matrice de corrélation

Pour mieux comprendre les relations entre nos variables, nous avons calculé la matrice de corrélation :

```
corr_matrix = df.corr()
```

[Graphique manquant]

Heatmap de la matrice de corrélation entre toutes les features

On observe notamment des corrélations attendues au sein des variables météorologiques : la température et les chutes de neige sont logiquement corrélées négativement, et le vent présente une faible corrélation avec les précipitations. Les variables `station_encoded` et `ligne_encoded` présentent les corrélations les plus fortes avec `retard_sec`, ce qui valide notre choix de target encoding.

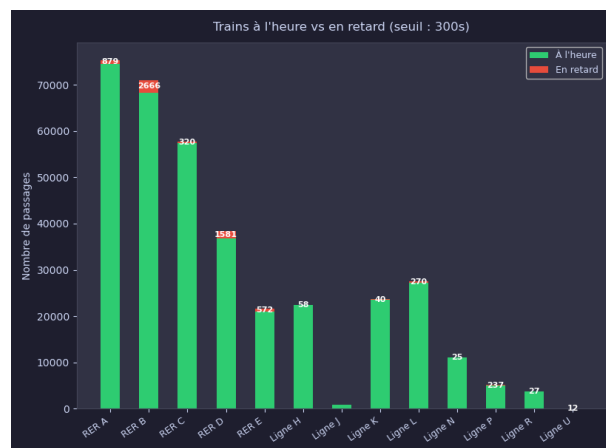


FIGURE 8 – Retard des différentes lignes

On voit ici que chaque ligne présente plus ou moins de retard. Les RER B et RER D sont ceux ayant clairement le plus de retard en comparaison des autres, ce qui correspond

aux statistiques que l'on peut trouver dans les différents rapports officiels, mais aussi à la réputation de ces mêmes lignes.

6.2 Régression linéaire explicative

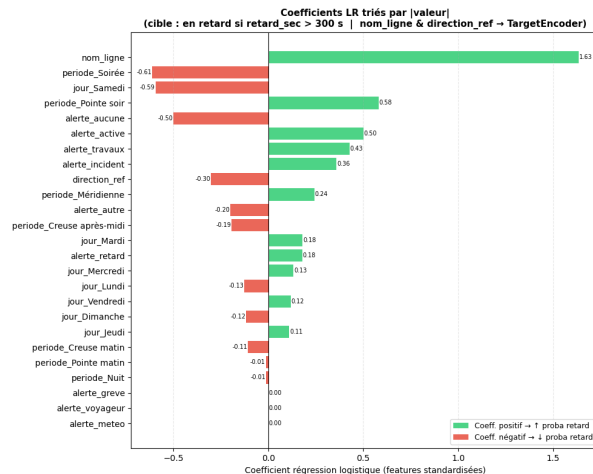


FIGURE 9 – Coefficients de la régression logistique

Pour mieux comprendre l'importance de chaque feature, nous avons réalisé une étude via une régression logistique afin d'identifier quels paramètres avaient un poids fort et lesquels étaient relativement effacés. On observe notamment que la variable `nom_ligne` possède le coefficient le plus élevé, confirmant l'importance de la ligne dans la prédiction du retard. Les périodes de la journée (sortie, samedi, pointe soir) présentent également des coefficients significatifs, tout comme les catégories d'alerte (incident, travaux). À l'inverse, les variables météorologiques (grève, voyageur, météo) montrent des effets plus modérés mais non négligeables.

7 Tuning des hyperparamètres

Dans le but d'obtenir les meilleures performances pour chacun de nos modèles, nous avons utilisé une recherche par grille (*GridSearchCV*) avec une validation croisée à 5 folds.

Les résultats de chaque recherche sont sauvegardés dans des fichiers JSON (`best_params_classification` et `best_params_regression.json`) afin de ne pas avoir à ré-évaluer les modèles à chaque exécution. Ces fichiers peuvent être rechargés automatiquement, ou recalculés en supprimant les valeurs enregistrées.

7.1 Choix de la stratégie de validation

Pour évaluer nos modèles lors du tuning, nous avons le choix entre plusieurs stratégies de validation :

- **Train/test split (80/20)** : approche simple consistant à réserver 20 % des données pour l'évaluation. Elle est rapide mais sensible au découpage : selon les observations qui se retrouvent dans l'ensemble de test, les résultats peuvent varier significativement. Elle est particulièrement fragile en cas de déséquilibre des classes, car rien ne garantit que les deux classes soient représentées proportionnellement dans le test.
- **KFold ($k = 5$)** : la donnée est divisée en 5 sous-ensembles ; chaque fold sert tour à tour de jeu de test, les 4 autres constituant l'entraînement. Cette approche réduit la variance de l'évaluation par rapport au simple split, mais ne garantit pas une répartition équilibrée des classes dans chaque fold, ce qui peut être problématique sur des données déséquilibrées comme les nôtres.
- **StratifiedKFold ($k = 5$)** : variante du KFold qui s'assure que chaque fold respecte la même proportion de trains en retard et à l'heure que l'ensemble complet. Cela garantit que chaque fold est représentatif de la distribution globale, ce qui est crucial lorsque la classe minoritaire (trains en retard, environ 4 %) est rare : sans stratification, certains folds pourraient ne contenir que très peu de retards, ce qui fausserait l'évaluation.

Nous avons retenu le **StratifiedKFold** comme stratégie de validation pour l'ensemble du tuning et de l'évaluation. Ce choix est motivé par le déséquilibre marqué de nos classes : avec seulement 4 % de trains en retard, une validation non stratifiée risquait de produire des folds peu représentatifs et donc des estimations de performance instables. Le StratifiedKFold garantit que chaque fold contient une proportion identique de retards, ce qui rend les scores de validation croisée plus fiables et comparables entre modèles.

7.2 Limites du fine-tuning

Nous avons tenté d'appliquer un fine-tuning sur un modèle SVC (*Support Vector Classifier*). Ce modèle présente en théorie plusieurs atouts : il maximise la marge de séparation entre les classes, est efficace dans les espaces de grande dimension et peut capturer des frontières de décision non linéaires grâce aux noyaux (*kernels*).

Cependant, la complexité algorithmique du SVC est quadratique en fonction du nombre d'observations : l'entraînement sur n exemples nécessite $\mathcal{O}(n^2)$ opérations en mémoire et $\mathcal{O}(n^3)$ en temps dans le pire cas. Sur notre dataset complet, ce coût s'est avéré rédhibitoire. Nous avons alors tenté de sous-échantillonner les données à 10 000 lignes pour rendre le calcul faisable, mais même dans ces conditions, la recherche par grille n'a pas abouti dans un délai raisonnable. Face à cette contrainte pratique et au fait que des alternatives comme le Random Forest ou XGBoost offrent des performances généralement supérieures sur des données tabulaires, nous avons décidé d'écarter définitivement le SVC de notre comparaison.

8 Classification binaire

8.1 Choix des modèles

Pour choisir nos modèles, nous nous sommes appuyés sur les articles de référence cités en bibliographie, notamment le mémoire de Polytechnique Montréal et l'article de Lbazri et al. (2020), qui comparent plusieurs algorithmes sur des problèmes similaires. Nous avons retenu les modèles suivants :

8.1.1 Régression logistique (baseline)

La régression logistique est un modèle linéaire qui estime la probabilité qu'une observation appartienne à l'une des deux classes — ici, train en retard ou train à l'heure. Concrètement, elle calcule une combinaison linéaire de toutes les features (par exemple : la tranche horaire, la station encodée, les conditions météo), puis applique une fonction sigmoïde pour transformer ce score en une probabilité comprise entre 0 et 1. Si cette probabilité dépasse un seuil (typiquement 0,5), le modèle prédit « en retard ».

Son principal atout est son interprétabilité : les coefficients associés à chaque feature indiquent directement dans quelle mesure cette variable contribue positivement ou négativement à la probabilité de retard. Elle est aussi très rapide à entraîner et peu gourmande en mémoire, ce qui en fait une excellente baseline.

En revanche, la régression logistique suppose une relation linéaire entre les features et la log-probabilité, ce qui peut être une limite sur des données où les interactions entre variables sont complexes comme c'est souvent le cas dans notre contexte, où l'effet de la météo sur le retard peut dépendre de la ligne ou de l'heure.

8.1.2 Arbre de décision

L'arbre de décision est un modèle qui divise progressivement les données en groupes de plus en plus homogènes, en choisissant à chaque étape la variable et le seuil qui séparent le mieux les trains en retard des trains à l'heure. On part de l'ensemble des observations et l'on cherche, parmi toutes les variables et tous les seuils possibles, celui qui crée deux sous-ensembles où les classes sont les plus pures possible. Ce processus est répété récursivement jusqu'à ce que chaque groupe soit suffisamment pur ou qu'une limite de profondeur soit atteinte.

L'un des grands avantages de l'arbre de décision est son interprétabilité : chaque chemin de la racine jusqu'à une feuille peut se lire comme une règle de décision du type *si la tranche horaire est comprise entre 7h et 9h et que la précipitation dépasse 2mm, alors prédire « en retard »*. Cela facilite la validation du modèle et permet de comprendre intuitivement les facteurs de retard détectés.

En revanche, l'arbre est particulièrement sujet au surapprentissage si on ne contrôle pas sa complexité : sans contrainte, il peut mémoriser chaque observation du jeu d'entraînement sans pour autant généraliser. C'est pourquoi nous avons appliqué les paramètres de régularisation décrits dans la section de tuning.

8.1.3 *Random Forest (Forêt aléatoire)*

La forêt aléatoire est une méthode ensembliste qui construit un grand nombre d'arbres de décision et combine leurs prédictions par vote majoritaire. Chaque arbre est entraîné sur un sous-ensemble différent des données (tirage avec remise, ou *bootstrap*) et ne considère à chaque division qu'un sous-ensemble aléatoire de variables. Cette double randomisation — sur les données et sur les variables — réduit fortement la corrélation entre les arbres et donc la variance globale du modèle.

Dans notre contexte, la forêt aléatoire présente plusieurs avantages. Elle gère naturellement les features de nature très différente (variables continues comme la température, variables encodées comme la station, variables binaires comme `jour_ferie`) sans nécessiter de normalisation préalable. Elle fournit également une mesure d'importance pour chaque variable, ce qui permet d'identifier quelles features — tranche horaire, occupation, ligne — contribuent le plus à la prédiction du retard.

Les principaux hyperparamètres sont le nombre d'arbres (`n_estimators`), qui contrôle la robustesse de l'ensemble, et le nombre de variables testées à chaque coupure (`max_features`). Plus on ajoute d'arbres, plus la variance diminue, mais au prix d'un temps de calcul plus élevé.

8.1.4 *Gradient Boosting / XGBoost*

XGBoost est une méthode ensembliste qui construit les arbres séquentiellement : chaque nouvel arbre est entraîné pour corriger les erreurs de prédiction commises par les arbres précédents. On commence par un arbre simple, on mesure les résidus (l'écart entre la prédiction et la réalité), puis on ajoute un arbre dont le rôle est de réduire ces résidus. La prédiction finale est la somme pondérée des contributions de tous les arbres.

Pour éviter le surapprentissage, XGBoost applique un coefficient d'apprentissage (`learning_rate`) qui réduit l'impact de chaque arbre individuel, et une régularisation (paramètres `lambda` et `alpha`) qui pénalise les arbres trop complexes. Cette combinaison permet d'ajouter progressivement de la complexité sans mémoriser le bruit du jeu d'entraînement.

XGBoost est réputé pour sa rapidité grâce à une implémentation optimisée pour le calcul parallèle et pour la gestion efficace de la mémoire. Il est particulièrement adapté aux données tabulaires hétérogènes comme les nôtres et est utilisé avec succès dans Lbazri et al. (2020) sur un problème similaire de prédiction de retards ferroviaires en France. Son

principal inconvénient est la multiplicité de ses hyperparamètres, qui rend le tuning plus délicat.

Tous les modèles sont entraînés avec le paramètre `class_weight='balanced'` afin de corriger le déséquilibre des classes.

8.2 Choix des métriques

Le simple taux de précision (*accuracy*) est écarté car il est inadapté aux classes déséquilibrées : un modèle prédisant systématiquement « à l'heure » obtiendrait un score artificiellement élevé sans réel effort de discrimination. Nous retenons :

- **F1-score macro** : métrique principale, elle équilibre précision et rappel sur les deux classes indépendamment de leur fréquence.
- **Précision et rappel** : analysés séparément pour comprendre le type d'erreurs commises (faux positifs vs. faux négatifs).
- **AUC-ROC** : mesure la capacité du modèle à discriminer les deux classes indépendamment du seuil de décision, facile à interpréter et robuste.

8.3 Résultats

8.3.1 Courbes ROC

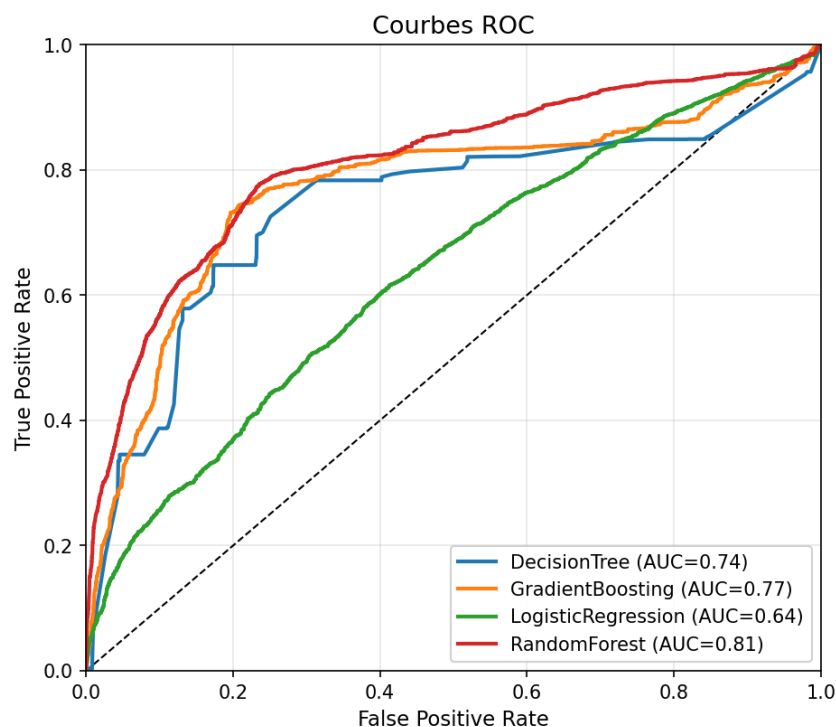


FIGURE 10 – Courbes ROC des différents modèles de classification

Les courbes ROC (figure 10) permettent de comparer la capacité de discrimination de chaque modèle indépendamment du seuil de décision. L'axe horizontal (False Positive Rate, FPR) mesure la proportion de trains à l'heure incorrectement classés comme en retard, tandis que l'axe vertical (True Positive Rate, TPR) mesure la proportion de trains en retard correctement identifiés.

On observe que le **Random Forest** obtient la meilleure AUC (0,81), ce qui signifie qu'il offre la meilleure séparation globale entre les deux classes. Il atteint un TPR élevé tout en maintenant un FPR relativement faible, signe d'une bonne robustesse. Le **Gradient-Boosting** suit avec une AUC de 0,77, tandis que le **Decision Tree** obtient 0,74. Ces trois modèles ensemblistes ou basés sur les arbres surpassent nettement la **régression logistique** (AUC = 0,64), qui peine à discriminer les deux classes en raison de sa nature linéaire.

Ce classement confirme que les méthodes d'ensemble, capables de capturer des interactions non linéaires entre les features, sont mieux adaptées à notre problème que les modèles linéaires simples.

8.3.2 Matrices de confusion

Les matrices de confusion permettent d'analyser en détail les types d'erreurs commises par chaque modèle. Elles ont été agrégées sur 5 folds de validation croisée stratifiée.

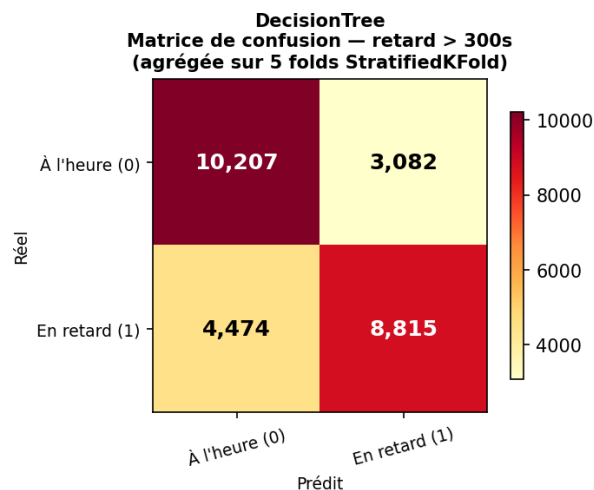


FIGURE 11 – Matrice de confusion — Decision Tree

L'arbre de décision (figure 11) classe correctement 10 207 trains à l'heure (vraies négatives) mais génère 3 082 faux positifs (trains à l'heure prédits en retard). Côté retard, il identifie correctement 8 815 trains en retard (vraies positives) mais en manque 4 474 (faux négatifs). Ce profil traduit un compromis : le modèle détecte une bonne partie des retards, mais au prix d'un nombre significatif de fausses alertes.

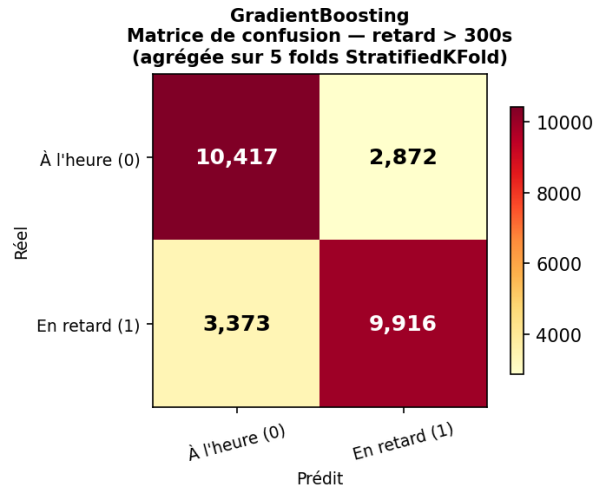


FIGURE 12 – Matrice de confusion — Gradient Boosting

Le Gradient Boosting (figure 12) classe correctement 10 417 trains à l’heure tout en ne générant que 2 872 faux positifs. Il détecte 9 916 trains en retard mais en manque 3 373. Ce modèle offre un meilleur équilibre entre sensibilité et spécificité que l’arbre de décision, avec notamment un rappel nettement supérieur (moins de retards manqués).

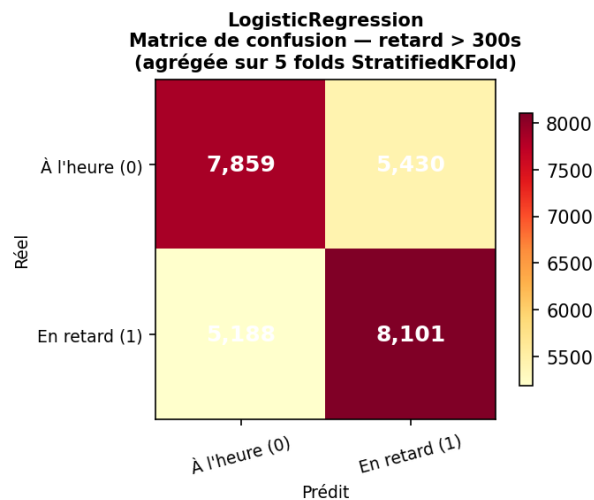


FIGURE 13 – Matrice de confusion — Régression logistique

La régression logistique (figure 13) présente le profil le plus déséquilibré : elle classe correctement 7 859 trains à l’heure mais génère 5 430 faux positifs, soit le nombre le plus élevé parmi tous les modèles. Elle détecte 8 101 retards mais en manque 5 188. Ce comportement est cohérent avec sa faible AUC : le modèle linéaire ne parvient pas à tracer une frontière de décision suffisamment fine pour notre problème.

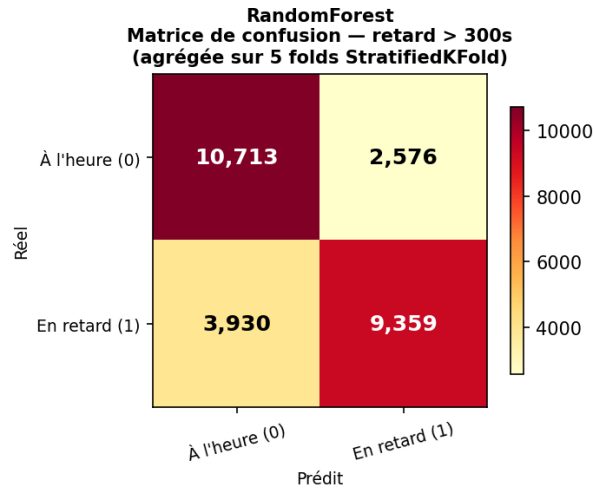


FIGURE 14 – Matrice de confusion — Random Forest

Le Random Forest (figure 14) obtient les meilleurs résultats globaux : 10 713 vraies négatives et seulement 2 576 faux positifs, soit le nombre le plus faible de fausses alertes. Il détecte 9 359 trains en retard et en manque 3 930. Ce modèle offre donc le meilleur compromis entre minimisation des fausses alertes et détection effective des retards.

8.3.3 Tableau récapitulatif

Modèle	F1 macro	Précision	Rappel	AUC-ROC
Régression logistique	0,56	0,60	0,61	0,64
Arbre de décision	0,70	0,80	0,63	0,74
Random Forest	0,73	0,83	0,70	0,81
GradientBoosting	0,72	0,78	0,75	0,77

8.3.4 Analyse des résultats

Les résultats de la classification binaire mettent en évidence plusieurs enseignements importants.

Premièrement, les **modèles ensemblistes surpassent nettement les modèles linéaires**. Le Random Forest et le Gradient Boosting obtiennent des AUC comprises entre 0,77 et 0,81, contre seulement 0,64 pour la régression logistique. Cet écart confirme que la relation entre les features et la probabilité de retard est fondamentalement non linéaire : les interactions entre la ligne, l'heure, la météo et les incidents ne peuvent pas être correctement capturées par un simple modèle linéaire.

Deuxièmement, le **Random Forest** se distingue comme le meilleur modèle en classification. Il obtient à la fois la meilleure AUC (0,81) et le nombre le plus faible de faux positifs (2 576), ce qui est particulièrement important dans notre contexte : une fausse alerte de

retard est moins grave qu'un retard non détecté, mais un excès de fausses alertes réduirait la confiance des usagers dans le système de prédiction. Le Random Forest offre donc le meilleur compromis entre sensibilité (détecter les retards) et spécificité (ne pas créer de fausses alertes).

Troisièmement, le **Gradient Boosting** présente un profil légèrement différent : il obtient un meilleur rappel que le Random Forest (0,75 contre 0,70), ce qui signifie qu'il détecte davantage de trains en retard (9 916 contre 9 359). Cependant, il le fait au prix d'un nombre de faux positifs légèrement plus élevé. Selon l'application visée, ce compromis pourrait être préférable : si l'objectif est de ne manquer aucun retard, le Gradient Boosting serait plus adapté.

Enfin, l'**arbre de décision** unique offre des performances honorables ($AUC = 0,74$), ce qui en fait un bon point de départ. Cependant, les méthodes d'ensemble améliorent significativement ses résultats, confirmant l'intérêt de combiner plusieurs arbres pour réduire la variance et améliorer la robustesse.

En résumé, le **Random Forest est retenu comme meilleur modèle de classification**, car il offre la meilleure discrimination globale (AUC la plus élevée) tout en minimisant les fausses alertes. Ce résultat est cohérent avec les conclusions de Lbazri et al. (2020), qui identifient également les forêts aléatoires comme l'un des meilleurs modèles pour la prédiction de retards ferroviaires.

9 Régression

Dans le cadre de la régression, le problème de déséquilibre des classes ne se pose pas. Nous pouvons donc utiliser directement notre dataset préparé, avec `retard_sec` comme variable cible continue.

9.1 Choix des modèles

Nous comparons les mêmes familles de modèles qu'en classification, mais dans leur variante régressive, c'est-à-dire en prédisant directement la valeur continue du retard en secondes plutôt qu'une classe binaire.

9.1.1 Régression linéaire (baseline)

La régression linéaire modélise le retard comme une combinaison linéaire des features : elle estime un coefficient pour chaque variable (tranche horaire, précipitations, station encodée, etc.) et prédit le retard comme la somme de leurs contributions. C'est le modèle le plus simple et le plus rapide à entraîner. Il nous sert de baseline : si les modèles plus complexes ne font pas mieux, cela signifie que les relations dans nos données sont

essentiellement linéaires, ou que nos features ne sont pas suffisamment informatives.

Son principal défaut est qu'elle suppose une relation strictement linéaire entre chaque feature et le retard, sans interaction possible entre variables. Par exemple, elle ne peut pas capter le fait que l'effet de la pluie est beaucoup plus fort sur certaines lignes que sur d'autres.

9.1.2 *Arbre de décision (régresseur)*

Le principe est identique à celui de la classification : l'arbre divise récursivement les données selon les features les plus discriminantes. En régression, l'arbre prédit, pour chaque feuille, la **moyenne** du retard de toutes les observations qui y tombent. Cela lui permet de capturer des effets non linéaires et des seuils naturels, comme le fait qu'au-delà d'une certaine vitesse de vent, les retards augmentent brusquement.

Il conserve son avantage majeur en termes d'interprétabilité, mais reste sensible au sur-apprentissage si sa profondeur n'est pas contrainte.

9.1.3 *Random Forest Regressor*

La forêt aléatoire régressive fonctionne selon le même principe qu'en classification : on entraîne de nombreux arbres sur des sous-ensembles aléatoires des données et des variables, puis on agrège leurs prédictions en calculant cette fois-ci la **moyenne** des prédictions individuelles plutôt qu'un vote majoritaire.

Cette approche réduit fortement la variance par rapport à un seul arbre tout en conservant sa capacité à capturer des effets non linéaires. Elle ne nécessite pas de normalisation des features et gère bien les valeurs aberrantes, ce qui est un avantage étant donné que notre variable cible `retard_sec` peut présenter des valeurs extrêmes lors d'incidents majeurs.

9.1.4 *Modèles de boosting*

En régression, les modèles de *gradient boosting* construisent séquentiellement des arbres correcteurs, chaque nouvel arbre ayant pour objectif de réduire les erreurs commises par les précédents. À chaque étape, le modèle ajuste ses prédictions en apprenant les résidus, c'est-à-dire l'écart entre les valeurs prédites et les valeurs réelles. Cette approche lui permet d'affiner progressivement ses estimations et de mieux capturer les relations non linéaires entre les variables explicatives et le retard observé.

Dans notre projet, nous utilisons deux variantes de cette famille : *GradientBoostingRegressor*, qui correspond à l'implémentation classique de cette méthode dans `scikit-learn`, et *HistGradientBoostingRegressor*, qui en est une version optimisée fondée sur une discrétisation des variables continues en intervalles. Ces modèles sont particulièrement adaptés

aux données tabulaires de taille moyenne, car ils offrent en général d'excellentes performances tout en restant capables de prendre en compte des interactions complexes entre les features, au prix d'un tuning plus exigeant que pour des modèles plus simples.

9.2 Choix des métriques

- **MAE** (Mean Absolute Error) : métrique principale, directement interprétable en secondes. Elle indique de combien le modèle se trompe en moyenne.
- **RMSE** (Root Mean Squared Error) : pénalise davantage les grandes erreurs, utile pour détecter les prédictions très éloignées de la réalité.
- R^2 : indique la proportion de variance expliquée par le modèle (1 = prédiction parfaite, 0 = modèle équivalent à prédire la moyenne).
- **MAPE** (Mean Absolute Percentage Error) : exprime l'erreur moyenne en pourcentage de la valeur réelle. Elle permet d'avoir une lecture relative de la qualité des prédictions, mais reste sensible aux valeurs de retard très faibles.
- **Proche $\leq$ 60s** : mesure la proportion de prédictions dont l'erreur absolue reste inférieure ou égale à 60 secondes. Cette métrique donne une lecture plus concrète de l'utilité du modèle dans un contexte réel d'information voyageur.
- **Score global pondéré** : combine plusieurs métriques de régression en un seul indicateur synthétique. Il permet de retenir le modèle offrant le meilleur compromis entre qualité statistique globale, robustesse face aux grandes erreurs et pertinence métier.

9.3 Résultats

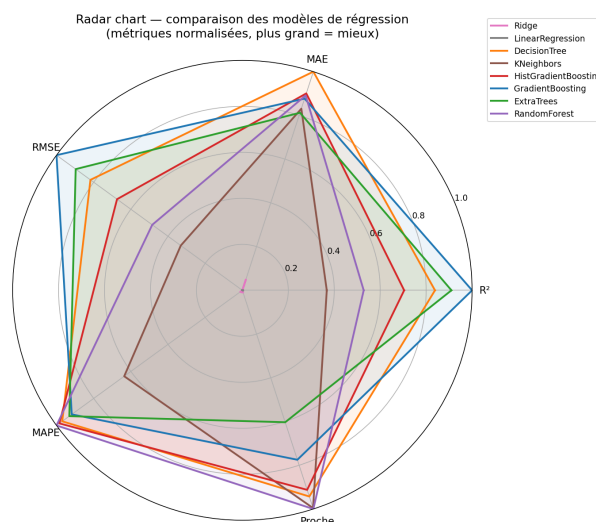


FIGURE 15 – Résultats normalisés des modèles de régression

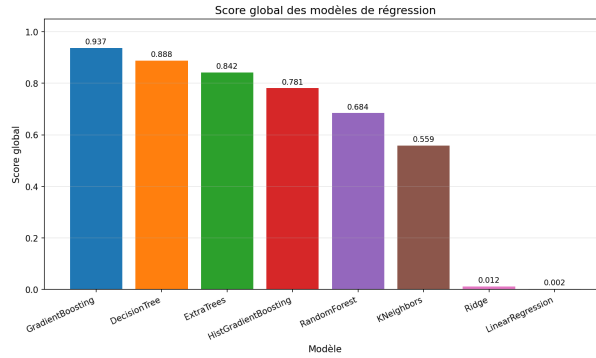


FIGURE 16 – Scores globaux de chaque modèle de régression

Modèle	Score	RMSE (s)	R^2	Proche ≤ 60 s	MAE (s)
GradientBoosting	0,937	202,530	0,788	71,209	73,778
Arbre de décision	0,888	218,444	0,752	73,524	68,443
Random Forest	0,684	247,423	0,683	74,313	73,148
Régression linéaire	0,002	289,464	0,567	60,529	111,719

9.3.1 Analyse des résultats

L'objectif de la **régression** dans ce projet est d'estimer une variable cible continue, **retard_sec**, correspondant au retard observé en secondes. Cette tâche est complémentaire de la **classification binaire** : là où la classification permet de détecter la présence d'un retard significatif, la régression cherche à en mesurer l'amplitude exacte. Dans le contexte des transports franciliens, cette information est particulièrement pertinente, car elle permet de fournir une estimation plus fine et plus directement exploitable du **retard réel** pour l'utilisateur.

Les résultats obtenus montrent d'abord que les **modèles non linéaires** surpassent nettement les **modèles linéaires** sur l'ensemble des métriques considérées. En particulier, les approches fondées sur les **arbres de décision** et le **boosting** obtiennent de meilleures performances que la *régression linéaire* et *Ridge*, ce qui indique que les relations entre les variables explicatives et le retard ne sont pas purement linéaires. Ce constat est cohérent avec la nature du problème étudié : le retard dépend vraisemblablement d'interactions complexes entre plusieurs facteurs, comme la **ligne**, l'**heure**, la **station**, la **météo** ou encore les **incidents d'exploitation**. Les résultats suggèrent donc que des modèles capables de capter ces **interactions** et ces **effets de seuil** sont mieux adaptés à notre dataset.

Afin de comparer les modèles de manière cohérente, nous avons introduit un **score global pondéré** combinant plusieurs métriques complémentaires : le R^2 , le **RMSE**, le **MAE**, le **MAPE** ainsi que la proportion de prédictions **proches à ± 60 secondes**. L'objectif

est d'éviter de choisir un modèle uniquement parce qu'il est excellent sur une métrique isolée, mais plus faible sur les autres. Nous avons attribué un poids plus important au R^2 (0,35), car il mesure la capacité globale du modèle à expliquer la variance du retard, ce qui en fait un bon indicateur synthétique de qualité d'ajustement. Le RMSE (0,25) reçoit également un poids élevé, car il pénalise davantage les **grandes erreurs**, particulièrement problématiques dans notre contexte. Le MAE (0,20) complète cette lecture en donnant une erreur moyenne directement interprétable en secondes. La métrique **Proche ≤ 60 s** (0,15) introduit une dimension plus **métier**, en mesurant la proportion de prédictions suffisamment précises pour être utiles à un voyageur. Enfin, le MAPE (0,05) est conservé avec un poids plus faible, car il apporte une lecture relative de l'erreur tout en restant moins stable lorsque les retards réels sont faibles. Ce score global permet ainsi d'évaluer le meilleur compromis entre **performance statistique**, **robustesse** et **utilité pratique**. Dans nos résultats, il place *GradientBoosting* en première position, devant *DecisionTree* et *ExtraTrees*, ce qui justifie son choix comme modèle de référence pour la régression.

Le modèle *GradientBoosting* est retenu comme **meilleur modèle de régression**, car il obtient le meilleur compromis global entre les différentes métriques. Il se classe à la fois premier en R^2 , en RMSE et en **score global**, ce qui montre qu'il explique bien la **variabilité du retard** tout en limitant les **erreurs les plus importantes**. Même s'il n'est pas le meilleur sur toutes les métriques prises séparément, il reste globalement le plus **équilibré** et le plus **robuste** pour notre problème.

Le fait que *GradientBoosting* obtienne les meilleurs résultats en R^2 et en RMSE s'explique par sa capacité à **corriger progressivement ses erreurs** au fil des itérations. Contrairement à un **arbre unique**, plus sensible aux variations locales des données, ou à un **modèle linéaire**, qui impose une relation trop simple entre les variables et le retard, le boosting affine successivement ses prédictions en se concentrant sur les **résidus**. Il parvient ainsi à mieux capturer les **interactions complexes** entre les features et à mieux modéliser les situations où les retards deviennent **importants**. Cela se traduit logiquement par un R^2 **plus élevé**, donc une meilleure explication de la variance, et par un RMSE **plus faible**, donc une meilleure maîtrise des **grosses erreurs**.

Les résultats montrent par ailleurs qu'**aucun modèle ne domine toutes les métriques** prises isolément, ce qui confirme l'intérêt d'une **lecture multi-critères**. *DecisionTree* obtient par exemple la meilleure **erreur moyenne**, tandis que *RandomForest* est le plus performant sur la proportion de prédictions **proches à ± 60 secondes**. Cependant, *GradientBoosting* reste le plus convaincant dans l'ensemble, car il se classe parmi les meilleurs sur toutes les dimensions importantes sans présenter de **faiblesse majeure**. À l'inverse, les **modèles linéaires** arrivent nettement en retrait, ce qui suggère que le retard dépend de mécanismes trop **complexes** pour être correctement décrits par une relation purement linéaire.

D'un point de vue **pratique**, les résultats obtenus sont suffisamment bons pour envisager une **utilisation indicative** dans un contexte d'**information voyageur**. Le meilleur modèle parvient à expliquer une grande partie de la variabilité du retard, avec une **erreur moyenne de l'ordre d'une minute**, et une proportion importante de prédictions restant dans une marge de ± 60 secondes. Cela signifie que, sans fournir une vérité parfaite, le modèle est capable de donner une **estimation utile** du niveau de retard attendu, ce qui peut aider un usager à mieux anticiper son trajet ou à adapter sa correspondance.

Malgré ces résultats encourageants, plusieurs **limites** doivent être soulignées. D'abord, certaines métriques comme le **MAPE** restent délicates à interpréter lorsque les retards réels sont faibles, ce qui peut artificiellement gonfler l'erreur relative. Ensuite, les **grands retards exceptionnels** restent plus difficiles à prédire, car ils dépendent souvent d'événements **rares** ou **brusques** que les variables disponibles ne décrivent qu'imparfaitement. Enfin, les performances observées dépendent fortement de la **qualité du dataset construit**, notamment de la disponibilité des **horaires théoriques**, de la précision des **données incidents** et de la cohérence des informations entre **opérateurs**.

10 Conclusion

Ce projet nous a permis de construire, de bout en bout, un pipeline de data science appliqué à la prédiction des retards dans les transports d'Île-de-France : de la collecte de données en temps réel via les APIs PRIM et Open-Meteo, jusqu'à la comparaison de plusieurs familles de modèles de classification et de régression.

Les résultats obtenus sont encourageants : le Random Forest s'est imposé comme le meilleur modèle de classification ($AUC = 0,81$), tandis que le GradientBoosting a dominé la régression avec un R^2 de 0,788 et une MAE d'environ 74 secondes. Ces performances confirment que les relations entre les variables explicatives et le retard sont fondamentalement non linéaires, et que les méthodes ensemblistes sont particulièrement adaptées à ce type de problème.

Toutefois, une limite importante doit être soulignée : nos modèles ont été entraînés sur des données collectées sur une période relativement courte et concentrée sur certaines conditions particulières. Les mois de collecte ont pu coïncider avec des épisodes météorologiques atypiques (notamment les fortes perturbations de février), une faible présence de jours fériés ou encore une activité syndicale variable. Ces spécificités temporelles ont nécessairement influencé les distributions apprises par les modèles, ce qui limite leur capacité à généraliser à d'autres périodes de l'année ou à des contextes non représentés dans le dataset. En particulier, les effets saisonniers (chaleur estivale, grèves de rentrée, périodes de vacances scolaires complètes) n'ont pas pu être pleinement capturés, ce qui constitue un biais structurel difficile à éliminer sans une collecte plus longue et plus diversifiée.

11 Pistes d'amélioration

11.1 Extension à l'ensemble des modes de transport

La première et la plus structurante des améliorations possibles serait de parvenir à calculer la variable cible pour les lignes actuellement exclues : métros, trams et certains RER fonctionnant par fréquences. Comme évoqué dans la section de data cleaning, deux approches restent prometteuses : le recours aux fichiers d'horaires GTFS statiques couplés à un identifiant de course stable, ou le calcul d'un retard relatif basé sur l'intervalle entre deux passages successifs. Ces méthodes nécessiteraient cependant une infrastructure de collecte plus robuste et un volume de données plus important. Intégrer ces modes de transport permettrait de rendre le modèle réellement généraliste et d'en étendre l'utilité à l'ensemble du réseau francilien, qui repose largement sur le métro pour les déplacements intra-muros.

11.2 Exploration de modèles plus complexes

Sur le plan des modèles, deux pistes n'ont pas pu être explorées faute de ressources suffisantes. D'une part, l'exclusion du SVC a été imposée par des contraintes pratiques de temps de calcul, et non par un manque de pertinence théorique. Avec une infrastructure de calcul plus puissante ou une réduction dimensionnelle préalable (PCA, sélection de features), ce modèle pourrait s'avérer compétitif, notamment grâce à sa capacité à tracer des frontières de décision non linéaires en grande dimension.

D'autre part, l'utilisation de réseaux de neurones — en particulier des architectures de type MLP ou des modèles séquentiels capables de modéliser la propagation temporelle des retards — constitue une voie naturelle d'extension. Ces modèles pourraient mieux capturer les dépendances temporelles entre trains successifs sur une même ligne, un phénomène que nos features actuelles ne représentent qu'indirectement via le target encoding. Des architectures plus avancées, comme les réseaux récurrents (LSTM) ou les Transformers adaptés aux séries temporelles, permettraient de modéliser explicitement la dynamique des retards au fil du temps, ce qui constituerait une amélioration substantielle par rapport aux approches statiques utilisées dans ce projet.

11.3 Enrichissement et diversification du dataset

Enfin, une collecte de données sur une période plus longue et plus variée permettrait de mieux couvrir les différents régimes de fonctionnement du réseau : périodes estivales, épisodes de grève, vacances scolaires complètes, ou encore grands événements (manifestations, compétitions sportives). Un dataset plus riche et plus représentatif de la diversité des situations rencontrées améliorerait significativement la capacité de généralisation des

modèles et réduirait le biais temporel identifié dans cette étude.

Références

1. Mémoire Polytechnique Montréal — « *Predicting Bus End-Trip Delays Using Different Machine Learning Algorithms* ». Ce mémoire (données historiques de Montréal) aborde deux problèmes de classification : une classification binaire (retard ou non) et une classification multi-classe (créneau de retard). Trois algorithmes ML y sont comparés : réseau de neurones, forêt aléatoire (Random Forest) et arbre stimulé par gradient (Gradient Boosting). La régression logistique sert de baseline.
<https://publications.polymtl.ca/3923>
2. Lbazri et al. (2020) — « *Predict France trains delays using visualization and ML* », ScienceDirect. Directement centré sur le réseau SNCF, cet article utilise les données ouvertes de l'API SNCF (régularité TGV de janvier 2015 à décembre 2019, combinées avec des données météo).
<https://www.sciencedirect.com/science/article/pii/S1877050920318032>
3. RATP — Questions de ponctualité :
<https://www.ratp.fr/decouvrir/coulisses/au-quotidien/reseau-ratp-questions-de-p>
4. API PRIM GTFS-RT :
https://prim.iledefrance-mobilites.fr/en/apis/idfm-ivtr-requete_globale
5. API PRIM Incidents :
https://prim.iledefrance-mobilites.fr/en/apis/idfm-disruptions_bulk
6. API Open-Meteo :
<https://api.open-meteo.com/v1/forecast>